- `backend/mcp/client.py`

Structured records are defined in:

- `backend/observability/tool_trace_context.py`
- `backend/observability/models.py`

Records are written in:

- `backend/metrics/collector.py`
- `metrics/chat_metrics.jsonl`

Observability is assembled in:

- `backend/main.py`

If you want, I can also make you a **cleaner "presentation-style" diagram** with color-style sections like:

- **Tracing**
- **Record definitions**
- **Persistence**
- **Analytics**

reading through the opentelemetry code inside the project, it appears like at some point it 'copies' the tools or something in order for the spans and traces to record. Is that correct? Can you walk me through what it does and how it works?

I'm going back to the patched files so I can explain the exact wrap/clone flow rather than just describe it abstractly.

Thought for 39s

Yes — in the **current** `client.py` **you switched to**, the code **does clone/copy the tool object**.

But one important clarification:

That is not OpenTelemetry itself doing the copying.
That is **your observability integration in** `backend/mcp/client.py` doing it so tracing can be added safely.

The short answer

The flow is:

1. load MCP tools normally
2. for each tool, build a **traced wrapper function**
3. create a **copied tool object** with the wrapped function/coroutine swapped in
4. give those wrapped tools to the agent
5. when the agent calls a tool, the wrapper opens the span and records the tool-call data

So yes, a “copied” version of the tool is what the agent ends up using.

Where this happens

The key file is:

```
backend/mcp/client.py
```

And the key function is:

Python Run

That is the place where the loaded tool gets turned into a traced version.

Why it copies instead of mutating

This is because your tools are `StructuredTool`-style objects, which are often Pydantic-backed.

Earlier, trying to do something like this:

Python Run

caused the error:

```
ValueError: "StructuredTool" object has no field "ainvoke"
```

So the fix was:

- do **not** mutate the tool object in place
- instead, create a **new copy** of the tool with updated fields like:
  - `func`
  - `coroutine`

That preserves the original tool structure while swapping in traced behavior.

What it is actually copying

It is **not** making a deep copy of the whole MCP server or duplicating the tool logic.

It is more like this:

Original tool

```
StructuredTool
- name
- description
- args schema
- func or coroutine
```

Wrapped tool

```
StructuredTool
- same name
- same description
- same args schema
- new func/coroutine that calls the original one inside a trace span
```

So the only thing really changing is:

```
execution entrypoint
```

The metadata stays the same.

Step-by-step walkthrough

1. OTEL setup is created once

In:

```
backend/observability/otel.py
```

your code does things like:

- create tracer provider
- choose exporter
- expose `get_tracer(...)`

That means later modules can do:

Python Run

This file does **not** copy tools.
It just provides the tracing machinery.

2. MCP tools are loaded from the gateway

In `backend/mcp/client.py`, `_ensure_alias_loaded()` does the actual loading:

```
cm = self.client.session(alias)
session = await cm.__aenter__()
tools = await load_mcp_tools(session)
wrapped_tools = [_wrap_tool(tool, alias=alias) for tool in tools]
```

So the sequence is:

- get MCP session
- load tool objects from that session
- pass each tool into `_wrap_tool(...)`

This is the moment where the tool objects get replaced by traced versions.

3. `_wrap_tool()` inspects the tool

Inside `_wrap_tool(...)`, the code checks for the callable parts of the tool:

- `func`
- `coroutine`

Depending on whether the tool is sync or async, it creates a traced wrapper around that callable.

Conceptually:

```
original_func = getattr(tool, "func", None)
original_coroutine = getattr(tool, "coroutine", None)
```

Then it builds replacements.

4. The wrapper function is created

For sync tools, `_wrap_sync_callable(...)` returns a function that does roughly this:

```
def traced_func(*args, **kwargs):
    start span
    record tool metadata
    call original_func(*args, **kwargs)
    record duration / success / output size
    append ToolCallRecord
    return result
```

For async tools, `_wrap_async_callable(...)` does the same thing, but with `await`.

So these wrappers are the actual place where tracing happens for tool execution.

5. The tool is cloned with updated callables

This is the “copy” step you noticed.

The code uses something like:

Python Run

or:

Python Run

depending on what the tool object supports.

That means:

- make a new tool object
- keep all existing fields
- replace only the execution field (`func` or `coroutine`)

So yes — this is the exact place where the tool is copied.

6. The wrapped tool is cached and later handed to the agent

After wrapping, `_ensure_alias_loaded()` stores the wrapped tools:

Python Run

Then `get_all_tools()` or `get_tools_by_names()` returns those wrapped copies, and the agent is built using them.

So from that point forward, the agent is **not using the raw original tool object**.
It is using the traced wrapped copy.

7. During a request, tool records are stored per request

In `backend/agent/engine.py` , before the agent runs, the code starts a request-scoped collection:

Python Run

That comes from:

```
backend/observability/tool_trace_context.py
```

This creates an empty request-local list using a `ContextVar`.

Then when a wrapped tool runs, the wrapper adds a typed record:

```python
append_tool_call_record(
    ToolCallRecord(...)
)
```

So the wrapper does two things at runtime:

- emits the child span immediately
- saves a structured record for later summary

8. After the agent turn, the engine summarizes those records

Still in `backend/agent/engine.py`, after `self.agent.ainvoke(...)` finishes, the engine gets the current request's tool records:

```python
tool_records = get_tool_call_records()
tool_summary = summarize_tool_records()
```

Then it puts the summary onto the parent span, for example:

- total tool count
- total tool duration
- max tool duration
- tool names

So the child tool spans come from the wrapper, and the parent summary comes later from the collected records.

The clean mental model

Think of it like this:

`otel.py`

Provides the tracing system.

`client.py`

Injects tracing into tools by replacing the tool's callable with a wrapped one.

`tool_trace_context.py`

Stores request-local tool call records while the request is running.

`engine.py`

Reads those records afterward and summarizes them onto the parent span.

A simple pseudo-flow

```
load raw MCP tool
    ↓
build traced wrapper around original func/coroutine
    ↓
clone tool with wrapped callable
    ↓
agent uses cloned wrapped tool
    ↓
tool gets called
    ↓
wrapper starts span and records ToolCallRecord
    ↓
engine summarizes all tool records after request ends
```

Important distinction

So if your question is:

> "Does OpenTelemetry in this project copy the tools?"

The most precise answer is:

Not OpenTelemetry itself.
Your tracing integration in `backend/mcp/client.py` copies/clones the tool objects so it can safely insert traced callables.

Why this is a good design

Because it gives you a single instrumentation point for all tools without editing every MCP server.

That means:

- no changes needed in each tool server
- tool metadata stays intact
- the agent still sees a normal tool object
- tracing happens automatically whenever the tool executes

That is why this approach fits your architecture well.

If you want, I can draw a tiny code-level sequence diagram just for the "tool clone → tool execute → record summary" path.

So to be clear, this does NOT create more tool calls and latency with it copying?

Thought for a couple of seconds

Correct — it does **not** create more MCP tool calls.

It creates a **wrapped copy of the tool object in memory**, but when that wrapped tool is executed, it still calls the **same original underlying tool function/coroutine exactly once**.

So:

- **extra MCP/network tool calls:** no
- **extra agent decisions:** no

- **extra round trips to the gateway:** no
- **some extra local Python overhead:** yes, a little

What actually happens

Without tracing:

```
agent -> tool -> result
```

With your wrapped copy:

```
agent -> wrapped tool -> original tool -> result
```

That wrapper just does a few local things:

- start a span
- capture a timestamp
- call the original tool once
- record duration / status / sizes
- append one in-memory record

So the only added latency is the small cost of that bookkeeping.

The important distinction

It is copying the **tool object definition**, not duplicating the **tool execution**.

Think of it like replacing this:

Python Run

```python
tool.func = original_func
```

with a copied tool whose function is effectively:

Python Run

```python
def traced_func(...):
    start_trace()
    result = original_func(...)
    record_trace()
    return result
```

That still runs `original_func(...)` one time.

Will there be any latency impact?

Yes, but usually small.

The added cost is mostly:

- Python wrapper call overhead
- `time.perf_counter()`
- span creation/attribute setting
- appending one record to a request-local list
- possibly console export overhead if `ConsoleSpanExporter` is enabled

So the biggest possible slowdown in your current setup is probably **console exporting**, not the tool copy itself.

Good rule of thumb

The copy step happens when tools are loaded, not every time the user sends a prompt.

So:

- **startup/load time:** tiny extra cost to wrap/clone tools
- **per request:** tiny extra tracing overhead
- **per tool call:** still only one real tool execution

So the direct answer is:

No, it does not create extra tool calls. It only adds a thin tracing wrapper around the existing single tool call, which adds a small amount of local latency.